CLAUDE.md — Módulo de Seguros Agéntico para atiende.ai

GUÍA DEFINITIVA PARA CLAUDE CODE

Replicando Handle AI para el Mercado Mexicano de Seguros

~100 Páginas | Abril 2026

ÍNDICE

PARTE 1: VISIÓN Y ARQUITECTURA (Páginas 1-15)

- Cap 1: Qué estamos construyendo — visión completa
- Cap 2: Handle AI — qué hacen y cómo lo replicamos
- Cap 3: Arquitectura general del sistema
- Cap 4: Stack tecnológico completo

PARTE 2: BASE DE DATOS Y MODELOS (Páginas 16-28)

- Cap 5: Schema de base de datos (Supabase/PostgreSQL)
- Cap 6: Modelos de datos de seguros
- Cap 7: Multi-tenancy y aislamiento de datos

PARTE 3: MOTOR DE BROWSER AUTOMATION AGÉNTICO (Páginas 29-48)

- Cap 8: Arquitectura del motor agéntico
- Cap 9: Skyvern + Playwright — integración completa
- Cap 10: Workers por aseguradora (Qualitas, GNP, AXA, etc.)
- Cap 11: Credential vault y gestión de sesiones
- Cap 12: Circuit breakers, retries y health monitoring

PARTE 4: MULTICOTIZADOR INTELIGENTE (Páginas 49-62)

- Cap 13: Quote Orchestrator — fan-out/scatter-gather
- Cap 14: Normalización de cotizaciones heterogéneas
- Cap 15: Motor de comparación y ranking

- Cap 16: Caché inteligente de cotizaciones

PARTE 5: INTEGRACIÓN WHATSAPP + VOZ (Páginas 63-74)

- Cap 17: Flujo conversacional de cotización por WhatsApp
- Cap 18: Entrega progresiva de resultados
- Cap 19: Agente de voz para seguros (Retell AI)
- Cap 20: Templates de conversación por tipo de seguro

PARTE 6: CRM DE SEGUROS + DASHBOARD (Páginas 75-86)

- Cap 21: CRM nativo de seguros
- Cap 22: Gestión de pólizas y renovaciones
- Cap 23: Reconciliación automática de pagos
- Cap 24: Dashboard del agente de seguros

PARTE 7: DEPLOYMENT Y OPERACIONES (Páginas 87-96)

- Cap 25: Docker containers para workers
- Cap 26: Deploy en Vercel + Railway/Fly.io
- Cap 27: Monitoreo y alertas
- Cap 28: Mantenimiento de scrapers

PARTE 8: FASES DE IMPLEMENTACIÓN (Páginas 97-108)

- Fase 0-11: Implementación paso a paso

PARTE 1: VISIÓN Y ARQUITECTURA

Capítulo 1: Qué Estamos Construyendo

1.1 Propuesta de valor en una oración

Un agente de seguros conecta sus credenciales de portales de aseguradoras → nuestro sistema AI cotiza automáticamente en 15-75 aseguradoras simultáneamente via browser automation agéntico → entrega comparativa rankeada al cliente final por WhatsApp → sincroniza todo al

CRM → gestiona renovaciones, pagos y pólizas automáticamente — 24/7, sin intervención humana.

1.2 Los 4 productos del módulo de seguros

1. **Multicotizador Agéntico WhatsApp**: El cliente manda datos por WhatsApp → AI extrae la info → cotiza en 15+ aseguradoras → entrega comparativa en <90 segundos. $2,999-$6,999 MXN/mes.
2. **CRM de Seguros AI-Native**: Detecta deals de email/WhatsApp automáticamente, crea oportunidades, trackea pipeline. Con reconciliación automática contra portales de aseguradoras. $1,499-$3,999 MXN/mes.
3. **Gestor de Renovaciones**: Monitorea 100% de pólizas próximas a vencer, alerta al agente, re-cotiza automáticamente, contacta al cliente. $999-$1,999 MXN/mes.
4. **Cobrador de Primas**: Accede a portales de aseguradoras, verifica status de pagos, envía recordatorios por WhatsApp, reconcilia contra CRM. $999-$1,999 MXN/mes.

1.3 El diferenciador vs Handle AI

Handle AI cobra $50K-$100K USD/año y requiere Forward Deployed Engineers. Nosotros lo ofrecemos como módulo self-service dentro de atiende.ai a $2,999-$6,999 MXN/mes (~$150-$350 USD/mes), con onboarding guiado por WhatsApp y zero deployment humano. Handle sirve brokers grandes en US/MX. Nosotros servimos a los 50,000+ agentes individuales y PyMEs de seguros en México que no pueden pagar $50K/año.

1.4 Mercado target

- **Primario**: Agentes de seguros individuales en México (~50,000 registrados en CNSF)
- **Secundario**: Agencias/brokerages pequeñas (2-20 agentes)
- **Terciario**: Empresas que quieren ofrecer seguros embedded (como add-on de atiende.ai)

1.5 Métricas de éxito

- Cotización completa en <90 segundos (vs 45-60 min manual)
- 80%+ tasa de éxito en portal automation
- 100% de renovaciones detectadas a tiempo
- Reducción de 94% en tiempo de registro de siniestros
- 40% más rápido en respuesta a clientes

Capítulo 2: Handle AI — Qué Hacen y Cómo lo Replicamos

2.1 Componentes de Handle que replicamos

Componente Handle	Nuestra versión	Diferencia clave
Signal (ingesta multi-canal)	WhatsApp + Email parser	WhatsApp-first en vez de email-first
AI Agents (browser automation)	Skyvern + Playwright workers	Open-source, self-hosted, costo 90% menor
Agent Builder (no-code)	Template system por vertical	Pre-configurado para seguros MX
Dashboard	Next.js + shadcn/ui	Integrado en atiende.ai existente
CRM sync	Supabase + API bidireccional	Nativo, no requiere Salesforce
Quoting automation	Fan-out orchestrator	Optimizado para aseguradoras MX
Payment tracking	Portal scraper + reconciliador	Específico para portales MX
Renewal management	Cron + alertas WhatsApp	Proactivo vía WhatsApp

2.2 Lo que NO replicamos (por ahora)

- Forward Deployed Engineers (somos self-service)
- Integración con Salesforce/HubSpot/Oracle (usamos CRM nativo)
- Multi-idioma (solo español MX)
- Claims automation complejo (v2)

2.3 Stack técnico de Handle (confirmado) vs nuestro stack

Capa	Handle	Nosotros
Frontend	React + TypeScript	Next.js 15 + TypeScript + shadcn/ui
Backend	TypeScript (Node) + Python	Next.js API Routes + Python (FastAPI)
ML/AI	PyTorch, Hugging Face, RAG	OpenRouter (Gemini 2.5 Flash) + Claude API

Capa	Handle	Nosotros
Database	PostgreSQL (prob. hosted)	Supabase (PostgreSQL + pgvector)
Real-time	WebSocket/SSE	Supabase Realtime + SSE
Browser automation	Playwright (inferido)	Skyvern Cloud + Playwright fallback
Cloud	AWS/GCP (inferido)	Vercel + Railway + Supabase
Credential mgmt	Custom vault	Supabase Vault + encryption at rest

Capítulo 3: Arquitectura General del Sistema

3.1 Diagrama de arquitectura completo

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CANALES DE ENTRADA                          │
│                                                                          │
│  WhatsApp (Meta Cloud API)     Email (Gmail/Outlook)     Dashboard Web   │
│  "Quiero cotizar seguro        intake@agencia.com         app.atiende.ai │
│   de auto para Jetta 2022"     con attachments             formulario    │
│                                                                          │
└──────────┬──────────────────────────┬──────────────────────────┬─────────┘
           │                          │                          │
           └──────────────┬───────────┴──────────────────────────┘
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                         INTAKE & EXTRACTION ENGINE                       │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐   │
│  │ WhatsApp Parser  │  │ Email Parser     │  │ Form Normalizer      │   │
│  │ (conversacional) │  │ (attachments)    │  │ (structured input)   │   │
│  └──────────┬───────┘  └──────────┬───────┘  └──────────┬───────────┘   │
│             └──────────────┬──────┴─────────────────────┘               │
│                            ▼                                             │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ AI Data Extractor (LLM)                                          │   │
│  │ Input: texto libre → Output: JSON estructurado                   │   │
│  │ {marca, modelo, año, CP, nombre, RFC, uso, cobertura_deseada}    │   │
│  └──────────────────────────┬───────────────────────────────────────┘   │
│                             │                                            │
```

```
│   ┌──────────────────────────────────────────────────────────────────┐    │
│   │  Validation Engine                                                │    │
│   │  - Verifica campos requeridos por tipo de seguro                  │    │
│   │  - Valida CP existe en catálogo SEPOMEX                           │    │
│   │  - Valida RFC formato (persona física/moral)                      │    │
│   │  - Si faltan datos → pregunta al usuario por WhatsApp             │    │
│   └──────────────────────────────────────────────────────────────────┘    │
└──────────────────────────────┬──────────────────────────────────────────┘
                               ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                    QUOTE ORCHESTRATOR (Fan-out Engine)                    │
│                                                                           │
│  Recibe: QuoteRequest (JSON validado)                                     │
│  Ejecuta: Fan-out a N carriers simultáneamente                            │
│                                                                           │
│   ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐        │
│   │Qualitas │  │  GNP    │  │  AXA    │  │  HDI    │  │  Chubb  │  ...x15+  │
│   │ Worker  │  │ Worker  │  │ Worker  │  │ Worker  │  │ Worker  │        │
│   └────┬────┘  └────┬────┘  └────┬────┘  └────┬────┘  └────┬────┘        │
│        │            │            │            │            │                │
│   ┌────▼────┐  ┌────▼────┐  ┌────▼────┐  ┌────▼────┐  ┌────▼────┐        │
│   │Skyvern  │  │Skyvern  │  │Skyvern  │  │Skyvern  │  │  API    │        │
│   │Browser  │  │Browser  │  │Browser  │  │Browser  │  │ Direct  │        │
│   │Agent    │  │Agent    │  │Agent    │  │Agent    │  │(BBVA)   │        │
│   └────┬────┘  └────┬────┘  └────┬────┘  └────┬────┘  └────┬────┘        │
│        │            │            │            │            │                │
│   ┌────▼────────────▼────────────▼────────────▼────────────▼────┐         │
│   │              RESPONSE NORMALIZER                            │         │
│   │  Cada carrier devuelve datos en formato diferente.          │         │
│   │  Normaliza a: {carrier, premium, deducible, cobertura,      │         │
│   │                vigencia, pdf_url, timestamp}                │         │
│   └──────────────────────────┬──────────────────────────────────┘         │
│                              │                                            │
│   ┌──────────────────────────▼──────────────────────────────────┐         │
│   │              RANKING ENGINE                                 │         │
│   │  Ordena por: precio (40%), cobertura (30%),                 │         │
│   │              carrier_rating (20%), velocidad_resp (10%)     │         │
│   │  Output: ranked_quotes[]                                    │         │
│   └──────────────────────────┬──────────────────────────────────┘         │
└──────────────────────────────┼──────────────────────────────────────────┘
                               ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                    DELIVERY & STORAGE                                     │
│                                                                           │
```

```
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐      │
│  │ WhatsApp Delivery │  │ Dashboard Display │  │ Supabase Store   │      │
│  │ - Progressive     │  │ - Tabla comparativa│  │ - quotes table   │      │
│  │ - Interactive list │  │ - PDFs descargables│  │ - PDFs en S3     │      │
│  │ - PDF attachment  │  │ - Filtros/sort    │  │ - audit log      │      │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                    BACKGROUND AGENTS (Always Running)                    │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐   │
│  │ Renewal Agent    │  │ Payment Agent    │  │ Reconciliation Agent │   │
│  │ Cron: diario     │  │ Cron: cada 6hrs  │  │ Cron: diario         │   │
│  │ Detecta pólizas  │  │ Accede portales  │  │ Compara portales     │   │
│  │ próximas a vencer│  │ verifica pagos   │  │ vs CRM, detecta      │   │
│  │ Alerta por WA    │  │ actualiza status │  │ discrepancias        │   │
│  └──────────────────┘  └──────────────────┘  └──────────────────────┘   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

3.2 Flujo de datos end-to-end (ejemplo: cotización auto)

```
1. Cliente WhatsApp: "Hola, quiero cotizar seguro de mi Jetta 2022"
2. AI extrae: {intent: "quote", line: "auto", partial_data: {marca: "VW", modelo:
"Jetta", año: 2022}}
3. AI pregunta faltantes: "¡Claro! Necesito unos datos más:
   - ¿Cuál es tu código postal?
   - ¿El auto es de uso particular o comercial?
   - ¿Qué tipo de cobertura prefieres? (Amplia / Limitada / RC)"
4. Cliente responde: "97000, particular, amplia"
5. Sistema valida: CP 97000 = Mérida, Yucatán ✓
6. Quote Orchestrator: Fan-out a 15 carriers
7. Progressive delivery: "⏳ Cotizando con 15 aseguradoras..."
8. A los 15s: "✅ Qualitas: $8,450/año | GNP: $9,120/año (2 de 15 listas)"
9. A los 45s: "✅ 12 de 15 cotizaciones listas"
10. A los 75s: Entrega comparativa final con tabla + PDF
11. CRM: Crea deal automáticamente con todas las cotizaciones
12. Seguimiento: Si no compra en 48hrs, follow-up automático por WhatsApp
```

Capítulo 4: Stack Tecnológico Completo

4.1 Stack definitivo

Componente	Tecnología	Versión	Razón
Frontend	Next.js + App Router	15.x	Ya usado en atiende.ai, SSR, Vercel-native
UI Components	shadcn/ui + Tailwind CSS	latest	Consistente con atiende.ai existente
Backend API	Next.js API Routes	15.x	Monorepo, server actions, edge functions
Quoting Engine	Python + FastAPI	3.12 / 0.115	Async nativo, ideal para orchestration
Browser Automation	Skyvern SDK + Playwright	latest / 1.x	AI-guided, self-healing, open-source
Database	Supabase (PostgreSQL 15)	latest	pgvector, RLS, Realtime, Vault, ya en uso
Queue System	Upstash Redis + BullMQ	latest	Fan-out, retries, dead-letter queues
LLM Primary	Gemini 2.5 Flash (OpenRouter)	latest	Rápido, barato, bueno para extraction
LLM Classifier	GPT-5-nano (OpenRouter)	latest	Intent detection, routing
LLM Complex	Claude Sonnet (Anthropic API)	4	Análisis de pólizas, comparación compleja
WhatsApp	Meta Cloud API	v21.0	Directo, sin intermediario, gratis
Voice	Retell AI + Telnyx	latest	Español MX natural, SIP trunk MX
File Storage	Supabase Storage (S3)	-	PDFs de pólizas, cotizaciones
PDF Processing	pdf-lib + Textract	-	Generar comparativas, extraer datos de pólizas
Credential Vault	Supabase Vault + AES-256	-	Credenciales de portales encriptadas
Cron Jobs	Vercel Cron + Upstash QStash	-	Renovaciones, pagos, reconciliación

Componente	Tecnología	Versión	Razón
Deploy Frontend	Vercel	-	Zero config, preview deploys
Deploy Workers	Railway (Docker)	-	Long-running browser agents
Monitoring	Sentry + Upstash	-	Errors, performance, browser agent health

4.2 Variables de entorno requeridas

```env
```

```
# === CORE (ya existentes en atiende.ai) ===
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
OPENROUTER_API_KEY=
WHATSAPP_TOKEN=
WHATSAPP_PHONE_NUMBER_ID=
WHATSAPP_VERIFY_TOKEN=

# === MÓDULO SEGUROS - NUEVAS ===
# Skyvern (browser automation)
SKYVERN_API_KEY=
SKYVERN_API_URL=https://api.skyvern.com/api/v1
# Alternativa: self-hosted
# SKYVERN_API_URL=http://localhost:8000/api/v1

# Redis (queue system)
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

# Claude API (análisis complejo de pólizas)
ANTHROPIC_API_KEY=

# Encryption key for credential vault
CREDENTIAL_ENCRYPTION_KEY=  # 32-byte hex key: openssl rand -hex 32

# Worker config
QUOTE_TIMEOUT_MS=120000          # 2 min max per carrier
QUOTE_CONCURRENCY=8              # Max simultaneous browser agents
QUOTE_CACHE_TTL_HOURS=4          # Cache duration for identical quotes

# Carrier portal credentials (encrypted in Supabase Vault, NOT here)
# Se configuran por tenant en el dashboard

# QStash (cron/scheduling)
QSTASH_TOKEN=
QSTASH_CURRENT_SIGNING_KEY=
QSTASH_NEXT_SIGNING_KEY=

# Sentry monitoring
SENTRY_DSN=
SENTRY_AUTH_TOKEN=
```

```
# Railway (worker deployment)
RAILWAY_TOKEN=
```

4.3 Dependencias del proyecto

```json
// package.json additions (Next.js monorepo)
{
  "dependencies": {
    // Existing atiende.ai deps...

    // === NEW: Insurance module ===
    "@upstash/redis": "^1.28.0",
    "@upstash/qstash": "^2.5.0",
    "bullmq": "^5.1.0",
    "ioredis": "^5.3.0",
    "pdf-lib": "^1.17.1",
    "pdf-parse": "^1.1.1",
    "crypto-js": "^4.2.0",
    "zod": "^3.22.0",
    "date-fns": "^3.3.0",
    "p-limit": "^5.0.0",
    "p-retry": "^6.2.0",
    "p-timeout": "^6.1.0"
  }
}
```

```txt
```

```
# requirements.txt (Python workers - Railway)
fastapi==0.115.0
uvicorn[standard]==0.30.0
httpx==0.27.0
playwright==1.43.0
skyvern-sdk==1.0.0
redis==5.0.0
pydantic==2.6.0
python-dotenv==1.0.0
cryptography==42.0.0
Pillow==10.2.0
pdf2image==1.17.0
pytesseract==0.3.10
supabase==2.4.0
```

PARTE 2: BASE DE DATOS Y MODELOS

Capítulo 5: Schema de Base de Datos

5.1 Tablas nuevas para el módulo de seguros

NOTA: Estas tablas se AGREGAN al schema existente de atiende.ai. No reemplazan nada.

```sql
```

```sql
-- ============================================
-- MÓDULO DE SEGUROS: SCHEMA COMPLETO
-- Prefijo: ins_ para todas las tablas de seguros
-- ============================================

-- 1. CARRIERS (Aseguradoras)
CREATE TABLE ins_carriers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,                          -- "Qualitas", "GNP", "AXA"
  slug TEXT UNIQUE NOT NULL,                   -- "qualitas", "gnp", "axa"
  logo_url TEXT,
  portal_url TEXT,                             -- URL del portal del agente
  portal_type TEXT NOT NULL DEFAULT 'browser',  -- 'browser' | 'api' | 'email'
  api_base_url TEXT,                           -- Si tiene API directa
  supported_lines TEXT[] DEFAULT '{}',         -- {'auto','vida','gastos_medicos','
  market_share_auto DECIMAL(5,2),             -- % del mercado de auto
  avg_response_time_ms INTEGER,                -- Tiempo promedio de respuesta
  is_active BOOLEAN DEFAULT true,
  health_status TEXT DEFAULT 'healthy',        -- 'healthy' | 'degraded' | 'down'
  last_health_check TIMESTAMPTZ,
  failure_rate_24h DECIMAL(5,2) DEFAULT 0,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Seed data: Top 15 aseguradoras MX por market share auto
INSERT INTO ins_carriers (name, slug, portal_url, portal_type, supported_lines, mark
('Qualitas', 'qualitas', 'https://agentes.qualitas.com.mx', 'browser', '{auto}', 32.
('GNP Seguros', 'gnp', 'https://agentesgnp.com.mx', 'browser', '{auto,vida,gastos_me
('AXA Seguros', 'axa', 'https://portalagentes.axa.com.mx', 'browser', '{auto,vida,ga
('HDI Seguros', 'hdi', 'https://agentes.hdi.com.mx', 'browser', '{auto,vida,hogar}',
('Chubb Seguros', 'chubb', 'https://agentes.chubb.com.mx', 'browser', '{auto,vida,ga
('BBVA Seguros', 'bbva', 'https://api.bbva.com/insurance', 'api', '{auto,vida,hogar}
('Zurich Seguros', 'zurich', 'https://portalagentes.zurich.com.mx', 'browser', '{aut
('Mapfre', 'mapfre', 'https://agentes.mapfre.com.mx', 'browser', '{auto,vida,gastos_
('Seguros Atlas', 'atlas', 'https://portal.segurosatlas.com.mx', 'browser', '{auto,v
('AIG Seguros', 'aig', 'https://agentes.aig.com.mx', 'browser', '{auto,vida,negocio}
('Banorte Seguros', 'banorte', 'https://seguros.banorte.com/agentes', 'browser', '{a
('Afirme Seguros', 'afirme', 'https://agentes.afirme.com', 'browser', '{auto,vida}',
('SURA', 'sura', 'https://agentes.segurossura.com.mx', 'browser', '{auto,vida,gastos
('MetLife', 'metlife', 'https://agentes.metlife.com.mx', 'browser', '{vida,gastos_me
('Allianz', 'allianz', 'https://agentes.allianz.com.mx', 'browser', '{auto,vida,gast
```

```sql
-- 2. CARRIER CREDENTIALS (por tenant/agente)
-- Encriptadas con AES-256-GCM
CREATE TABLE ins_carrier_credentials (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id) ON DELETE CASCADE,
  encrypted_username TEXT NOT NULL,       -- AES-256-GCM encrypted
  encrypted_password TEXT NOT NULL,       -- AES-256-GCM encrypted
  encryption_iv TEXT NOT NULL,            -- Initialization vector
  agent_number TEXT,                      -- Número de agente en la aseguradora
  is_active BOOLEAN DEFAULT true,
  last_login_success TIMESTAMPTZ,
  last_login_error TEXT,
  login_failure_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(tenant_id, carrier_id)
);

-- RLS: Solo el tenant dueño puede ver sus credenciales
ALTER TABLE ins_carrier_credentials ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_own_credentials" ON ins_carrier_credentials
  FOR ALL USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 3. QUOTE REQUESTS (solicitudes de cotización)
CREATE TABLE ins_quote_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  contact_id UUID REFERENCES contacts(id),           -- Cliente en el CRM
  conversation_id UUID REFERENCES conversations(id),  -- Conversación de WhatsApp

  -- Datos del solicitante
  client_name TEXT NOT NULL,
  client_phone TEXT,
  client_email TEXT,
  client_rfc TEXT,
  client_birthdate DATE,
  client_gender TEXT,                     -- 'M' | 'F'
  client_zip_code TEXT NOT NULL,
  client_state TEXT,                      -- Auto-derivado del CP
  client_city TEXT,                       -- Auto-derivado del CP

  -- Datos del riesgo (para auto)
```

```
insurance_line TEXT NOT NULL,          -- 'auto' | 'vida' | 'gastos_medicos' | 'ho

-- Auto-specific fields
vehicle_brand TEXT,
vehicle_model TEXT,
vehicle_year INTEGER,
vehicle_version TEXT,                  -- "Style" | "Comfortline" | etc
vehicle_use TEXT DEFAULT 'particular', -- 'particular' | 'comercial' | 'uber'
vehicle_vin TEXT,
coverage_type TEXT,                    -- 'amplia' | 'limitada' | 'rc_obligatoria'

-- Vida-specific fields
sum_insured DECIMAL(12,2),
beneficiaries JSONB,

-- GMM-specific fields
gmm_plan_type TEXT,                   -- 'individual' | 'familiar'
gmm_family_members JSONB,
gmm_hospital_level TEXT,              -- 'basico' | 'medio' | 'alto'

-- Hogar-specific fields
property_type TEXT,                    -- 'casa' | 'departamento' | 'oficina'
property_value DECIMAL(12,2),
property_contents_value DECIMAL(12,2),

-- Status tracking
status TEXT NOT NULL DEFAULT 'pending',  -- 'pending' | 'validating' | 'quoting' |
carriers_targeted INTEGER DEFAULT 0,     -- Cuántas aseguradoras se intentaron
carriers_succeeded INTEGER DEFAULT 0,    -- Cuántas respondieron exitosamente
carriers_failed INTEGER DEFAULT 0,       -- Cuántas fallaron

-- Timing
started_at TIMESTAMPTZ,
first_result_at TIMESTAMPTZ,            -- Cuándo llegó la primera cotización
completed_at TIMESTAMPTZ,               -- Cuándo se completaron todas
expires_at TIMESTAMPTZ,                 -- Cuándo expiran las cotizaciones

-- Source
source TEXT DEFAULT 'whatsapp',          -- 'whatsapp' | 'web' | 'voice' | 'email'
raw_input TEXT,                          -- Texto original del usuario
extracted_data JSONB,                    -- Datos extraídos por AI

created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
```

```sql
);

-- Índices para búsqueda rápida
CREATE INDEX idx_quote_requests_tenant ON ins_quote_requests(tenant_id);
CREATE INDEX idx_quote_requests_status ON ins_quote_requests(status);
CREATE INDEX idx_quote_requests_created ON ins_quote_requests(created_at DESC);
CREATE INDEX idx_quote_requests_contact ON ins_quote_requests(contact_id);

-- RLS
ALTER TABLE ins_quote_requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_own_quotes" ON ins_quote_requests
  FOR ALL USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 4. QUOTES (cotizaciones individuales por carrier)
CREATE TABLE ins_quotes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quote_request_id UUID NOT NULL REFERENCES ins_quote_requests(id) ON DELETE CASCADE
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),

  -- Resultado
  status TEXT NOT NULL DEFAULT 'pending',  -- 'pending' | 'running' | 'success' | 'd

  -- Datos de la cotización
  annual_premium DECIMAL(12,2),            -- Prima anual total
  monthly_premium DECIMAL(12,2),           -- Prima mensual (si aplica)
  quarterly_premium DECIMAL(12,2),         -- Prima trimestral (si aplica)
  semiannual_premium DECIMAL(12,2),        -- Prima semestral (si aplica)
  deductible_amount DECIMAL(12,2),         -- Deducible
  deductible_percentage DECIMAL(5,2),      -- % de deducible
  coinsurance_percentage DECIMAL(5,2),     -- % de coaseguro

  -- Coberturas detalladas
  coverages JSONB,                         -- Array de coberturas con montos
  /* Ejemplo:
  [
    {"name": "Daños materiales", "sum_insured": 350000, "deductible": "5%"},
    {"name": "Robo total", "sum_insured": 350000, "deductible": "10%"},
    {"name": "Responsabilidad civil", "sum_insured": 3000000, "deductible": 0},
    {"name": "Gastos médicos ocupantes", "sum_insured": 200000, "deductible": 0},
    {"name": "Asistencia vial", "included": true},
    {"name": "Auto sustituto", "days": 15}
  ]
  */
```

```
  -- Metadata
  policy_number_preview TEXT,              -- Número de póliza/cotización
  valid_until DATE,                        -- Vigencia de la cotización
  pdf_url TEXT,                            -- URL del PDF de cotización (Supabase St
  screenshot_url TEXT,                     -- Screenshot del portal para auditoría

  -- Performance
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  duration_ms INTEGER,                     -- Tiempo que tomó obtener la cotización

  -- Error tracking
  error_message TEXT,
  error_type TEXT,                          -- 'timeout' | 'auth_failed' | 'portal_d
  retry_count INTEGER DEFAULT 0,

  -- Ranking
  rank_position INTEGER,                   -- Posición en el ranking (1 = mejor)
  rank_score DECIMAL(5,2),                -- Score compuesto (0-100)

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_quotes_request ON ins_quotes(quote_request_id);
CREATE INDEX idx_quotes_tenant ON ins_quotes(tenant_id);
CREATE INDEX idx_quotes_carrier ON ins_quotes(carrier_id);
CREATE INDEX idx_quotes_status ON ins_quotes(status);

ALTER TABLE ins_quotes ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_own_individual_quotes" ON ins_quotes
  FOR ALL USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 5. POLICIES (pólizas emitidas/activas)
CREATE TABLE ins_policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  contact_id UUID REFERENCES contacts(id),
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),
  quote_id UUID REFERENCES ins_quotes(id),

  -- Datos de la póliza
  policy_number TEXT NOT NULL,
```

```sql
  insurance_line TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active',    -- 'active' | 'pending_payment' | 'cance

  -- Vigencia
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  days_to_renewal INTEGER GENERATED ALWAYS AS (end_date - CURRENT_DATE) STORED,

  -- Financiero
  total_premium DECIMAL(12,2),
  payment_frequency TEXT,                   -- 'anual' | 'semestral' | 'trimestral'
  next_payment_date DATE,
  payment_status TEXT DEFAULT 'current',    -- 'current' | 'due' | 'overdue' | 'paid

  -- Datos del riesgo (JSON flexible por tipo)
  risk_data JSONB,

  -- Documentos
  policy_pdf_url TEXT,
  endorsements JSONB DEFAULT '[]',          -- Endosos

  -- Comisión
  commission_percentage DECIMAL(5,2),
  commission_amount DECIMAL(12,2),
  commission_paid BOOLEAN DEFAULT false,

  -- Renovación
  auto_renew BOOLEAN DEFAULT true,
  renewal_quote_id UUID,                    -- Si se re-cotizó
  renewal_notification_sent BOOLEAN DEFAULT false,

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_policies_tenant ON ins_policies(tenant_id);
CREATE INDEX idx_policies_contact ON ins_policies(contact_id);
CREATE INDEX idx_policies_renewal ON ins_policies(end_date);
CREATE INDEX idx_policies_status ON ins_policies(status);
CREATE INDEX idx_policies_days_renewal ON ins_policies(days_to_renewal);

ALTER TABLE ins_policies ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_own_policies" ON ins_policies
  FOR ALL USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));
```

```sql
-- 6. POLICY PAYMENTS (tracking de pagos)
CREATE TABLE ins_policy_payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  policy_id UUID NOT NULL REFERENCES ins_policies(id) ON DELETE CASCADE,

  payment_number INTEGER,                   -- Recibo 1 de 12, etc
  amount DECIMAL(12,2) NOT NULL,
  due_date DATE NOT NULL,
  paid_date DATE,
  status TEXT NOT NULL DEFAULT 'pending',  -- 'pending' | 'paid' | 'overdue' | 'canc

  -- Verificación automática desde portal
  verified_from_portal BOOLEAN DEFAULT false,
  portal_check_at TIMESTAMPTZ,
  portal_reference TEXT,                    -- Referencia del portal de la asegurador

  -- Notificaciones
  reminder_sent BOOLEAN DEFAULT false,
  reminder_sent_at TIMESTAMPTZ,
  overdue_notification_sent BOOLEAN DEFAULT false,

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_payments_policy ON ins_policy_payments(policy_id);
CREATE INDEX idx_payments_due ON ins_policy_payments(due_date);
CREATE INDEX idx_payments_status ON ins_policy_payments(status);

ALTER TABLE ins_policy_payments ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_own_payments" ON ins_policy_payments
  FOR ALL USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 7. CLAIMS (siniestros)
CREATE TABLE ins_claims (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  policy_id UUID NOT NULL REFERENCES ins_policies(id),
  contact_id UUID REFERENCES contacts(id),

  claim_number TEXT,                        -- Número asignado por aseguradora
  claim_type TEXT NOT NULL,                 -- 'colision' | 'robo' | 'danos_terceros
  status TEXT NOT NULL DEFAULT 'intake',    -- 'intake' | 'submitted' | 'in_review'
```

```sql
  -- Datos del siniestro
  incident_date TIMESTAMPTZ,
  incident_description TEXT,
  incident_location TEXT,
  incident_photos JSONB DEFAULT '[]',      -- URLs de fotos

  -- Documentos
  documents JSONB DEFAULT '[]',            -- [{name, url, type}]

  -- Portal tracking
  submitted_to_portal BOOLEAN DEFAULT false,
  portal_submission_at TIMESTAMPTZ,
  portal_claim_reference TEXT,

  -- Financiero
  estimated_amount DECIMAL(12,2),
  approved_amount DECIMAL(12,2),
  paid_amount DECIMAL(12,2),

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE ins_claims ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_own_claims" ON ins_claims
  FOR ALL USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 8. CARRIER HEALTH LOG (monitoreo de portales)
CREATE TABLE ins_carrier_health_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),

  check_type TEXT NOT NULL,                -- 'login' | 'quote' | 'payment_check' |
  status TEXT NOT NULL,                    -- 'success' | 'failure' | 'timeout' | 'c
  response_time_ms INTEGER,
  error_message TEXT,
  screenshot_url TEXT,                     -- Para debugging

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_health_carrier ON ins_carrier_health_log(carrier_id);
CREATE INDEX idx_health_created ON ins_carrier_health_log(created_at DESC);
```

```sql
-- 9. AUTOMATION RUNS (log de ejecuciones de agents)
CREATE TABLE ins_automation_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),

  run_type TEXT NOT NULL,                  -- 'quote' | 'payment_check' | 'renewal_c
  reference_id UUID,                       -- quote_id, policy_id, claim_id, etc.

  status TEXT NOT NULL DEFAULT 'running',  -- 'running' | 'success' | 'failure' | 't

  -- Skyvern tracking
  skyvern_task_id TEXT,
  skyvern_workflow_id TEXT,

  -- Steps log
  steps JSONB DEFAULT '[]',               -- [{step, status, screenshot_url, timesta

  -- Performance
  started_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ,
  duration_ms INTEGER,

  error_message TEXT,
  retry_count INTEGER DEFAULT 0,

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_runs_tenant ON ins_automation_runs(tenant_id);
CREATE INDEX idx_runs_carrier ON ins_automation_runs(carrier_id);
CREATE INDEX idx_runs_type ON ins_automation_runs(run_type);
CREATE INDEX idx_runs_status ON ins_automation_runs(status);

-- 10. VEHICLE CATALOG (catálogo AMIS de vehículos)
CREATE TABLE ins_vehicle_catalog (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  amis_key TEXT UNIQUE,                    -- Clave AMIS del vehículo
  brand TEXT NOT NULL,
  model TEXT NOT NULL,
  year INTEGER NOT NULL,
  version TEXT,
  vehicle_type TEXT,                        -- 'sedan' | 'suv' | 'pickup' | 'van' |
```

```sql
  engine_cc INTEGER,
  doors INTEGER,
  passengers INTEGER,
  estimated_value DECIMAL(12,2),

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_vehicles_brand_model ON ins_vehicle_catalog(brand, model, year);
CREATE INDEX idx_vehicles_amis ON ins_vehicle_catalog(amis_key);

-- 11. ZIP CODE CATALOG (catálogo SEPOMEX)
CREATE TABLE ins_zip_codes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  zip_code TEXT NOT NULL,
  settlement TEXT,                        -- Colonia
  municipality TEXT NOT NULL,             -- Municipio
  state TEXT NOT NULL,                    -- Estado
  city TEXT,
  zone TEXT,                              -- 'urbana' | 'suburbana' | 'rural'
  risk_zone TEXT,                         -- Zona de riesgo sísmico/inundación

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_zip_code ON ins_zip_codes(zip_code);
CREATE INDEX idx_zip_state ON ins_zip_codes(state);

-- 12. QUOTE CACHE (evitar re-cotizar lo mismo)
CREATE TABLE ins_quote_cache (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cache_key TEXT UNIQUE NOT NULL,         -- Hash de: carrier_id + insurance_line +
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),

  quote_data JSONB NOT NULL,              -- Cotización cacheada

  created_at TIMESTAMPTZ DEFAULT NOW(),
  expires_at TIMESTAMPTZ NOT NULL         -- TTL de 4 horas default
);

CREATE INDEX idx_cache_key ON ins_quote_cache(cache_key);
CREATE INDEX idx_cache_expires ON ins_quote_cache(expires_at);

-- Cleanup automático de cache expirado
```

```sql
CREATE OR REPLACE FUNCTION cleanup_expired_cache() RETURNS void AS $$
BEGIN
  DELETE FROM ins_quote_cache WHERE expires_at < NOW();
END;
$$ LANGUAGE plpgsql;

-- ============================================
-- VISTAS ÚTILES
-- ============================================

-- Vista: Pólizas próximas a renovar (30 días)
CREATE VIEW ins_policies_near_renewal AS
SELECT
  p.*,
  c.name as carrier_name,
  ct.name as client_name,
  ct.phone as client_phone
FROM ins_policies p
JOIN ins_carriers c ON p.carrier_id = c.id
LEFT JOIN contacts ct ON p.contact_id = ct.id
WHERE p.status = 'active'
  AND p.days_to_renewal BETWEEN 0 AND 30
ORDER BY p.days_to_renewal ASC;

-- Vista: Pagos vencidos
CREATE VIEW ins_overdue_payments AS
SELECT
  pp.*,
  p.policy_number,
  c.name as carrier_name,
  ct.name as client_name,
  ct.phone as client_phone
FROM ins_policy_payments pp
JOIN ins_policies p ON pp.policy_id = p.id
JOIN ins_carriers c ON p.carrier_id = c.id
LEFT JOIN contacts ct ON p.contact_id = ct.id
WHERE pp.status = 'overdue'
  OR (pp.status = 'pending' AND pp.due_date < CURRENT_DATE)
ORDER BY pp.due_date ASC;

-- Vista: Carrier health dashboard
CREATE VIEW ins_carrier_health AS
SELECT
  c.id,
```

```
  c.name,
  c.slug,
  c.health_status,
  c.failure_rate_24h,
  COUNT(CASE WHEN h.status = 'success' AND h.created_at > NOW() - INTERVAL '24 hours
  COUNT(CASE WHEN h.status = 'failure' AND h.created_at > NOW() - INTERVAL '24 hours
  AVG(CASE WHEN h.created_at > NOW() - INTERVAL '24 hours' THEN h.response_time_ms E
FROM ins_carriers c
LEFT JOIN ins_carrier_health_log h ON c.id = h.carrier_id
GROUP BY c.id, c.name, c.slug, c.health_status, c.failure_rate_24h;
```

PARTE 3: MOTOR DE BROWSER AUTOMATION AGÉNTICO

Capítulo 8: Arquitectura del Motor Agéntico

8.1 Principio fundamental

El motor agéntico es el corazón del sistema. A diferencia de Handle AI que construyó todo propietario, nosotros usamos **Skyvern** (open-source, backed by $7.5M funding) como la capa de browser automation + AI, con Playwright como fallback para portales bien conocidos donde un script determinístico es más fiable.

8.2 Estrategia de integración por carrier

```
TIER 1 — API Directa (más confiable, menos mantenimiento)
├── BBVA Seguros (API pública documentada)
└── Carriers que ofrezcan API en el futuro

TIER 2 — Playwright Determinístico (scripts custom por portal)
├── Qualitas (32.8% market share, worth custom script)
├── GNP (12.5%, alto volumen justifica script custom)
└── AXA (8.3%, alto volumen justifica script custom)

TIER 3 — Skyvern AI Agent (self-healing, adaptable)
├── HDI
├── Chubb
├── Zurich
├── Mapfre
├── Atlas
```

```
├── AIG
├── Banorte
├── Afirme
├── SURA
├── MetLife
├── Allianz
└── Todos los demás
```

Lógica: Los 3 carriers con mayor volumen (>60% del mercado auto) justifican scripts Playwright custom porque son más rápidos y 100% determinísticos. Para el resto (40% del mercado), Skyvern AI agents que se auto-adaptan a cambios de UI.

8.3 Worker Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    RAILWAY (Docker)                          │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  FastAPI App (port 8080)                             │    │
│  │                                                      │    │
│  │  POST /quote         → Ejecutar cotización           │    │
│  │  POST /check-payment → Verificar pago                │    │
│  │  POST /download-doc  → Descargar documento           │    │
│  │  POST /submit-claim  → Registrar siniestro           │    │
│  │  GET  /health        → Health check                  │    │
│  │                                                      │    │
│  │  ┌──────────────────────────────────────────────┐    │    │
│  │  │  Worker Pool (asyncio)                       │    │    │
│  │  │                                              │    │    │
│  │  │  ┌──────────┐ ┌──────────┐ ┌──────────┐      │    │    │
│  │  │  │Browser 1| |Browser 2| |Browser 3|      │    │    │
│  │  │  |(Chrome) | |(Chrome) | |(Chrome) |      │    │    │
│  │  │  └──────────┘ └──────────┘ └──────────┘      │    │    │
│  │  │  Concurrency: QUOTE_CONCURRENCY (8)          |    |    |
│  │  └──────────────────────────────────────────────┘    │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  Playwright browsers pre-installed                           │
│  Skyvern SDK pre-configured                                  |
│  Chrome + extensions for anti-detection                      │
└──────────────────────────────────────────────────────────────┘
```

8.4 Código: Base Worker Class

python

```python
# workers/base_worker.py

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional, Dict, Any
from datetime import datetime
import asyncio
import logging
from enum import Enum

logger = logging.getLogger(__name__)

class WorkerStatus(Enum):
    PENDING = "pending"
    RUNNING = "running"
    SUCCESS = "success"
    DECLINED = "declined"
    ERROR = "error"
    TIMEOUT = "timeout"

@dataclass
class QuoteRequest:
    """Datos normalizados para solicitar cotización"""
    request_id: str
    tenant_id: str
    carrier_slug: str
    insurance_line: str  # 'auto' | 'vida' | 'gmm' | 'hogar'

    # Cliente
    client_name: str
    client_birthdate: str
    client_gender: str
    client_zip_code: str
    client_rfc: Optional[str] = None

    # Auto-specific
    vehicle_brand: Optional[str] = None
    vehicle_model: Optional[str] = None
    vehicle_year: Optional[int] = None
    vehicle_version: Optional[str] = None
    vehicle_use: str = "particular"
    coverage_type: str = "amplia"
```

```python
    # Credentials (decrypted)
    portal_username: Optional[str] = None
    portal_password: Optional[str] = None
    agent_number: Optional[str] = None

@dataclass
class QuoteResult:
    """Resultado normalizado de una cotización"""
    request_id: str
    carrier_slug: str
    status: WorkerStatus

    # Si success
    annual_premium: Optional[float] = None
    monthly_premium: Optional[float] = None
    deductible_amount: Optional[float] = None
    deductible_percentage: Optional[float] = None
    coverages: Optional[list] = None
    policy_number_preview: Optional[str] = None
    valid_until: Optional[str] = None
    pdf_url: Optional[str] = None
    screenshot_url: Optional[str] = None

    # Performance
    duration_ms: Optional[int] = None

    # Si error
    error_message: Optional[str] = None
    error_type: Optional[str] = None


class BaseCarrierWorker(ABC):
    """Clase base para todos los workers de aseguradoras"""

    carrier_slug: str
    carrier_name: str
    portal_url: str
    timeout_ms: int = 120000  # 2 minutos default
    max_retries: int = 2

    def __init__(self):
        self.logger = logging.getLogger(f"worker.{self.carrier_slug}")

    @abstractmethod
```

```
    async def execute_quote(self, request: QuoteRequest) -> QuoteResult:
        """Ejecutar la cotización. Cada carrier implementa su propia lógica."""
        pass

    @abstractmethod
    async def check_payment_status(self, policy_number: str, credentials: dict) -> d
        """Verificar status de pago de una póliza"""
        pass

    @abstractmethod
    async def download_document(self, policy_number: str, doc_type: str, credentials
        """Descargar un documento (póliza, recibo, endoso)"""
        pass

    async def run_with_retry(self, request: QuoteRequest) -> QuoteResult:
        """Ejecutar con reintentos y timeout"""
        for attempt in range(self.max_retries + 1):
            try:
                start = datetime.now()
                result = await asyncio.wait_for(
                    self.execute_quote(request),
                    timeout=self.timeout_ms / 1000
                )
                result.duration_ms = int((datetime.now() - start).total_seconds() *
                return result

            except asyncio.TimeoutError:
                self.logger.warning(f"Timeout en intento {attempt + 1} para {self.ca
                if attempt == self.max_retries:
                    return QuoteResult(
                        request_id=request.request_id,
                        carrier_slug=self.carrier_slug,
                        status=WorkerStatus.TIMEOUT,
                        error_message=f"Timeout after {self.timeout_ms}ms",
                        error_type="timeout"
                    )

            except Exception as e:
                self.logger.error(f"Error en intento {attempt + 1} para {self.carrie
                if attempt == self.max_retries:
                    return QuoteResult(
                        request_id=request.request_id,
                        carrier_slug=self.carrier_slug,
                        status=WorkerStatus.ERROR,
```

```
                error_message=str(e),
                error_type="unknown"
            )

        # Wait before retry (exponential backoff)
        await asyncio.sleep(2 ** attempt)
```

8.5 Código: Skyvern Worker (para carriers Tier 3)

```python
```

```python
# workers/skyvern_worker.py

from skyvern import Skyvern
from .base_worker import BaseCarrierWorker, QuoteRequest, QuoteResult, WorkerStatus
import json

class SkyvernCarrierWorker(BaseCarrierWorker):
    """Worker genérico que usa Skyvern AI para cualquier portal de aseguradora"""

    def __init__(self, carrier_slug: str, carrier_name: str, portal_url: str):
        self.carrier_slug = carrier_slug
        self.carrier_name = carrier_name
        self.portal_url = portal_url
        self.skyvern = Skyvern(api_key=os.environ["SKYVERN_API_KEY"])
        super().__init__()

    async def execute_quote(self, request: QuoteRequest) -> QuoteResult:
        """Usar Skyvern para cotizar en cualquier portal"""

        prompt = self._build_quote_prompt(request)

        try:
            task = await self.skyvern.run_task(
                url=self.portal_url,
                prompt=prompt,
                data_extraction_schema={
                    "type": "object",
                    "properties": {
                        "annual_premium": {"type": "number", "description": "Prima a
                        "monthly_premium": {"type": "number", "description": "Prima
                        "deductible_percentage": {"type": "number", "description": "
                        "deductible_amount": {"type": "number", "description": "Mont
                        "coverages": {
                            "type": "array",
                            "items": {
                                "type": "object",
                                "properties": {
                                    "name": {"type": "string"},
                                    "sum_insured": {"type": "number"},
                                    "included": {"type": "boolean"}
                                }
                            },
                            "description": "Lista de coberturas incluidas"
```

```
                },
                "quote_number": {"type": "string", "description": "Número de
                "valid_until": {"type": "string", "description": "Fecha de v
            },
            "required": ["annual_premium"]
        },
        wait_for_completion=True,
        max_steps=30,
        totp_verification_url=None,
        totp_identifier=None
    )

    if task.status == "completed" and task.extracted_information:
        data = task.extracted_information
        return QuoteResult(
            request_id=request.request_id,
            carrier_slug=self.carrier_slug,
            status=WorkerStatus.SUCCESS,
            annual_premium=data.get("annual_premium"),
            monthly_premium=data.get("monthly_premium"),
            deductible_amount=data.get("deductible_amount"),
            deductible_percentage=data.get("deductible_percentage"),
            coverages=data.get("coverages", []),
            policy_number_preview=data.get("quote_number"),
            valid_until=data.get("valid_until"),
            screenshot_url=task.screenshot_url if hasattr(task, 'screenshot_
        )
    else:
        return QuoteResult(
            request_id=request.request_id,
            carrier_slug=self.carrier_slug,
            status=WorkerStatus.ERROR,
            error_message=f"Skyvern task failed: {task.status}",
            error_type="portal_error"
        )

except Exception as e:
    return QuoteResult(
        request_id=request.request_id,
        carrier_slug=self.carrier_slug,
        status=WorkerStatus.ERROR,
        error_message=str(e),
        error_type="skyvern_error"
    )
```

```
def _build_quote_prompt(self, request: QuoteRequest) -> str:
    """Construir el prompt para Skyvern según el tipo de seguro"""

    if request.insurance_line == "auto":
        return f"""
        Necesito cotizar un seguro de auto en este portal de aseguradora.

        CREDENCIALES DE LOGIN:
        - Usuario: {request.portal_username}
        - Contraseña: {request.portal_password}
        - Número de agente: {request.agent_number or 'N/A'}

        PASOS:
        1. Iniciar sesión con las credenciales proporcionadas
        2. Navegar a la sección de cotización de autos / nueva cotización
        3. Llenar el formulario con estos datos:
           - Marca: {request.vehicle_brand}
           - Modelo: {request.vehicle_model}
           - Año: {request.vehicle_year}
           - Versión: {request.vehicle_version or 'La más básica disponible'}
           - Uso: {request.vehicle_use}
           - Tipo de cobertura: {request.coverage_type}
           - Código postal: {request.client_zip_code}
           - Nombre del asegurado: {request.client_name}
           - Fecha de nacimiento: {request.client_birthdate}
           - Género: {request.client_gender}
           - RFC: {request.client_rfc or 'Dejar vacío si es opcional'}
        4. Calcular/cotizar
        5. Extraer TODOS los datos de la cotización: prima anual, prima mensual,
        6. Si hay opción de descargar PDF, descargar

        IMPORTANTE:
        - Si hay campos que no tengo datos, dejar vacíos o seleccionar la opción
        - Si pide seleccionar de un dropdown y no encuentras la opción exacta, s
        - Si hay un CAPTCHA, intentar resolverlo
        - Si pide aceptar términos y condiciones, aceptarlos
        - Extraer TODAS las coberturas listadas con sus montos
        """

    # Agregar más tipos de seguro aquí...
    return ""

async def check_payment_status(self, policy_number: str, credentials: dict) -> d
```

```python
        """Verificar status de pago via Skyvern"""
        task = await self.skyvern.run_task(
            url=self.portal_url,
            prompt=f"""
            Iniciar sesión con:
            - Usuario: {credentials['username']}
            - Contraseña: {credentials['password']}

            Buscar la póliza número: {policy_number}
            Encontrar el status de pago actual.
            Extraer: monto pendiente, fecha de vencimiento, recibos pagados vs pendi
            """,
            data_extraction_schema={
                "type": "object",
                "properties": {
                    "payment_status": {"type": "string"},
                    "amount_due": {"type": "number"},
                    "due_date": {"type": "string"},
                    "receipts_paid": {"type": "integer"},
                    "receipts_total": {"type": "integer"}
                }
            },
            wait_for_completion=True
        )
        return task.extracted_information or {}

    async def download_document(self, policy_number: str, doc_type: str, credentials
        """Descargar documento via Skyvern"""
        # Implementar con Skyvern file download capability
        pass
```

8.6 Código: Playwright Worker (para carriers Tier 2 - Qualitas ejemplo)

python

```python
# workers/playwright/qualitas_worker.py

from playwright.async_api import async_playwright, Page
from ..base_worker import BaseCarrierWorker, QuoteRequest, QuoteResult, WorkerStatus
import asyncio

class QualitasWorker(BaseCarrierWorker):
    """Worker determinístico para Qualitas - 32.8% del mercado auto"""

    carrier_slug = "qualitas"
    carrier_name = "Qualitas"
    portal_url = "https://agentes.qualitas.com.mx"
    timeout_ms = 90000  # 90 segundos (Qualitas es rápida)

    async def execute_quote(self, request: QuoteRequest) -> QuoteResult:
        async with async_playwright() as p:
            browser = await p.chromium.launch(
                headless=True,
                args=['--no-sandbox', '--disable-setuid-sandbox']
            )
            context = await browser.new_context(
                viewport={'width': 1920, 'height': 1080},
                user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53
            )
            page = await context.new_page()

            try:
                # Step 1: Login
                await page.goto(self.portal_url, wait_until='networkidle')
                await page.fill('#username', request.portal_username)
                await page.fill('#password', request.portal_password)
                await page.click('#loginBtn')
                await page.wait_for_load_state('networkidle')

                # Verificar login exitoso
                if await page.query_selector('.error-message'):
                    error_text = await page.text_content('.error-message')
                    return QuoteResult(
                        request_id=request.request_id,
                        carrier_slug=self.carrier_slug,
                        status=WorkerStatus.ERROR,
                        error_message=f"Login failed: {error_text}",
                        error_type="auth_failed"
```

```
    )

# Step 2: Navegar a cotización
await page.click('text=Nueva Cotización')
await page.wait_for_load_state('networkidle')

# Step 3: Llenar datos del vehículo
# NOTA: Los selectores reales se deben validar contra el portal actu
# Esto es un template que se ajustará durante la implementación
await page.select_option('#brand', label=request.vehicle_brand)
await asyncio.sleep(1)  # Esperar carga de modelos
await page.select_option('#model', label=request.vehicle_model)
await asyncio.sleep(1)
await page.select_option('#year', value=str(request.vehicle_year))

if request.vehicle_version:
    await page.select_option('#version', label=request.vehicle_versi

await page.select_option('#use', value=request.vehicle_use)
await page.fill('#zipCode', request.client_zip_code)

# Step 4: Datos del conductor
await page.fill('#driverName', request.client_name)
await page.fill('#driverBirthdate', request.client_birthdate)
await page.select_option('#driverGender', value=request.client_gende

# Step 5: Tipo de cobertura
coverage_map = {
    'amplia': 'AMPLIA',
    'limitada': 'LIMITADA',
    'rc_obligatoria': 'RC'
}
await page.click(f'text={coverage_map.get(request.coverage_type, "AM

# Step 6: Cotizar
await page.click('#quoteBtn')
await page.wait_for_selector('.quote-result', timeout=30000)

# Step 7: Extraer resultados
annual_premium = await self._extract_number(page, '.annual-premium')
monthly_premium = await self._extract_number(page, '.monthly-premium
deductible = await self._extract_number(page, '.deductible')

# Extraer coberturas
```

```
            coverages = await self._extract_coverages(page)

            # Step 8: Screenshot para auditoría
            screenshot = await page.screenshot()
            # TODO: Upload screenshot to Supabase Storage

            # Step 9: Descargar PDF si disponible
            pdf_url = None
            download_btn = await page.query_selector('.download-pdf')
            if download_btn:
                async with page.expect_download() as download_info:
                    await download_btn.click()
                download = await download_info.value
                pdf_path = await download.path()
                # TODO: Upload PDF to Supabase Storage

            return QuoteResult(
                request_id=request.request_id,
                carrier_slug=self.carrier_slug,
                status=WorkerStatus.SUCCESS,
                annual_premium=annual_premium,
                monthly_premium=monthly_premium,
                deductible_amount=deductible,
                coverages=coverages,
                pdf_url=pdf_url
            )

        except Exception as e:
            # Tomar screenshot de error
            try:
                await page.screenshot(path=f"/tmp/error_{self.carrier_slug}_{req
            except:
                pass
            raise e

        finally:
            await browser.close()

    async def _extract_number(self, page: Page, selector: str) -> float:
        """Extraer un número de un elemento del DOM"""
        try:
            text = await page.text_content(selector)
            if text:
                # Limpiar formato MX: "$8,450.00" → 8450.0
```

```
                cleaned = text.replace('$', '').replace(',', '').replace(' ', '').st
                return float(cleaned)
        except:
            pass
        return None

    async def _extract_coverages(self, page: Page) -> list:
        """Extraer tabla de coberturas"""
        coverages = []
        rows = await page.query_selector_all('.coverage-row')
        for row in rows:
            name = await row.query_selector('.coverage-name')
            amount = await row.query_selector('.coverage-amount')
            if name:
                coverages.append({
                    "name": await name.text_content(),
                    "sum_insured": await self._extract_number_from_element(amount) i
                    "included": True
                })
        return coverages

    async def check_payment_status(self, policy_number: str, credentials: dict) -> d
        """Verificar pagos en portal Qualitas"""
        # Implementar similar al flujo de cotización
        pass

    async def download_document(self, policy_number: str, doc_type: str, credentials
        """Descargar documentos de Qualitas"""
        # Implementar
        pass
```

Capítulo 9: Quote Orchestrator — Fan-out/Scatter-Gather

9.1 Código: Quote Orchestrator

```typescript
```

```typescript
// src/lib/insurance/quote-orchestrator.ts

import { createClient } from '@supabase/supabase-js'
import { Redis } from '@upstash/redis'
import pLimit from 'p-limit'
import crypto from 'crypto'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
})

interface QuoteRequestInput {
  tenantId: string
  contactId?: string
  conversationId?: string
  insuranceLine: 'auto' | 'vida' | 'gastos_medicos' | 'hogar' | 'negocio'
  clientData: {
    name: string
    phone?: string
    email?: string
    rfc?: string
    birthdate?: string
    gender?: string
    zipCode: string
  }
  riskData: Record<string, any>  // Datos específicos del tipo de seguro
  source: 'whatsapp' | 'web' | 'voice' | 'email' | 'api'
  rawInput?: string
}

interface ProgressUpdate {
  requestId: string
  carriersTotal: number
  carriersCompleted: number
  carriersFailed: number
  results: Array<{
    carrierName: string
```

```
    carrierSlug: string
    status: string
    premium?: number
  }>
}

export class QuoteOrchestrator {
  private concurrencyLimit = pLimit(
    parseInt(process.env.QUOTE_CONCURRENCY || '8')
  )

  /**
   * Ejecutar cotización multi-carrier
   * Returns: quote_request_id para tracking
   */
  async executeQuote(input: QuoteRequestInput): Promise<string> {
    // 1. Crear el QuoteRequest en DB
    const { data: quoteRequest, error } = await supabase
      .from('ins_quote_requests')
      .insert({
        tenant_id: input.tenantId,
        contact_id: input.contactId,
        conversation_id: input.conversationId,
        client_name: input.clientData.name,
        client_phone: input.clientData.phone,
        client_email: input.clientData.email,
        client_rfc: input.clientData.rfc,
        client_birthdate: input.clientData.birthdate,
        client_gender: input.clientData.gender,
        client_zip_code: input.clientData.zipCode,
        insurance_line: input.insuranceLine,
        ...this.extractRiskFields(input.insuranceLine, input.riskData),
        status: 'validating',
        source: input.source,
        raw_input: input.rawInput,
        extracted_data: input.riskData,
        started_at: new Date().toISOString(),
      })
      .select('id')
      .single()

    if (error) throw new Error(`Failed to create quote request: ${error.message}`)

    const requestId = quoteRequest.id
```

```
  // 2. Obtener carriers disponibles para este tipo de seguro
  const carriers = await this.getAvailableCarriers(input.tenantId, input.insurance

  // 3. Actualizar status
  await supabase
    .from('ins_quote_requests')
    .update({
      status: 'quoting',
      carriers_targeted: carriers.length,
    })
    .eq('id', requestId)

  // 4. Crear registros de quotes individuales (pending)
  const quoteRecords = carriers.map(carrier => ({
    quote_request_id: requestId,
    tenant_id: input.tenantId,
    carrier_id: carrier.id,
    status: 'pending',
  }))

  await supabase.from('ins_quotes').insert(quoteRecords)

  // 5. Fan-out: ejecutar todos los carriers en paralelo
  // Esto se hace de forma async — no bloqueamos
  this.fanOutToCarriers(requestId, input, carriers)
    .catch(err => console.error('Fan-out error:', err))

  return requestId
}

/**
 * Fan-out a todos los carriers con concurrency limit
 */
private async fanOutToCarriers(
  requestId: string,
  input: QuoteRequestInput,
  carriers: any[]
) {
  const results = await Promise.allSettled(
    carriers.map(carrier =>
      this.concurrencyLimit(async () => {
        // Check cache first
        const cached = await this.checkCache(carrier.id, input)
```

```
        if (cached) {
          await this.saveQuoteResult(requestId, carrier, cached)
          await this.publishProgress(requestId)
          return cached
        }

        // Check circuit breaker
        if (await this.isCircuitOpen(carrier.slug)) {
          await this.markQuoteSkipped(requestId, carrier.id, 'circuit_open')
          return null
        }

        // Execute via worker API
        try {
          const result = await this.callWorker(carrier, input, requestId)
          await this.saveQuoteResult(requestId, carrier, result)
          await this.updateCircuitBreaker(carrier.slug, result.status === 'success

          // Cache successful results
          if (result.status === 'success') {
            await this.cacheResult(carrier.id, input, result)
          }

          // Publish progress for progressive delivery
          await this.publishProgress(requestId)

          return result
        } catch (err) {
          await this.markQuoteFailed(requestId, carrier.id, err.message)
          await this.updateCircuitBreaker(carrier.slug, false)
          await this.publishProgress(requestId)
          return null
        }
      })
    )
  )

  // All carriers done — finalize
  await this.finalizeQuoteRequest(requestId)
}

/**
 * Llamar al worker API en Railway
 */
```

```
private async callWorker(carrier: any, input: QuoteRequestInput, requestId: string
  const workerUrl = process.env.WORKER_API_URL || 'https://insurance-workers.up.ra

  // Decrypt credentials
  const credentials = await this.getDecryptedCredentials(input.tenantId, carrier.i

  const response = await fetch(`${workerUrl}/quote`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.WORKER_API_SECRET}`
    },
    body: JSON.stringify({
      request_id: requestId,
      carrier_slug: carrier.slug,
      carrier_portal_url: carrier.portal_url,
      carrier_portal_type: carrier.portal_type,
      insurance_line: input.insuranceLine,
      client_data: input.clientData,
      risk_data: input.riskData,
      credentials: credentials,
    }),
    signal: AbortSignal.timeout(parseInt(process.env.QUOTE_TIMEOUT_MS || '120000')
  })

  if (!response.ok) {
    throw new Error(`Worker returned ${response.status}: ${await response.text()}`
  }

  return await response.json()
}

/**
 * Publicar progreso via Redis pub/sub para progressive delivery
 */
private async publishProgress(requestId: string) {
  const { data: quotes } = await supabase
    .from('ins_quotes')
    .select('*, ins_carriers(name, slug)')
    .eq('quote_request_id', requestId)

  const progress: ProgressUpdate = {
    requestId,
    carriersTotal: quotes?.length || 0,
```

```
    carriersCompleted: quotes?.filter(q => q.status === 'success').length || 0,
    carriersFailed: quotes?.filter(q => ['error', 'timeout', 'declined'].includes(
    results: quotes?.filter(q => q.status === 'success').map(q => ({
      carrierName: q.ins_carriers.name,
      carrierSlug: q.ins_carriers.slug,
      status: q.status,
      premium: q.annual_premium,
    })) || [],
  }

  // Publish to Redis for SSE/WebSocket consumers
  await redis.publish(`quote:${requestId}`, JSON.stringify(progress))

  // Also store latest state
  await redis.set(`quote:${requestId}:state`, JSON.stringify(progress), { ex: 3600
}

/**
 * Circuit breaker: si un carrier falla >50% en las últimas 24h, skip
 */
private async isCircuitOpen(carrierSlug: string): Promise<boolean> {
  const key = `circuit:${carrierSlug}`
  const data = await redis.get(key) as any
  if (!data) return false

  const { failures, total } = data
  return total >= 5 && (failures / total) > 0.5
}

private async updateCircuitBreaker(carrierSlug: string, success: boolean) {
  const key = `circuit:${carrierSlug}`
  const data = (await redis.get(key) as any) || { failures: 0, total: 0 }

  data.total++
  if (!success) data.failures++

  await redis.set(key, data, { ex: 86400 }) // 24h TTL
}

/**
 * Cache de cotizaciones idénticas
 */
private async checkCache(carrierId: string, input: QuoteRequestInput) {
  const cacheKey = this.buildCacheKey(carrierId, input)
```

```
  const { data } = await supabase
    .from('ins_quote_cache')
    .select('quote_data')
    .eq('cache_key', cacheKey)
    .gt('expires_at', new Date().toISOString())
    .single()

  return data?.quote_data || null
}

private buildCacheKey(carrierId: string, input: QuoteRequestInput): string {
  const hashInput = JSON.stringify({
    carrierId,
    line: input.insuranceLine,
    risk: input.riskData,
    zip: input.clientData.zipCode,
    gender: input.clientData.gender,
    birthdate: input.clientData.birthdate,
  })
  return crypto.createHash('sha256').update(hashInput).digest('hex')
}

/**
 * Finalizar el quote request y rankear resultados
 */
private async finalizeQuoteRequest(requestId: string) {
  // Get all successful quotes
  const { data: quotes } = await supabase
    .from('ins_quotes')
    .select('*')
    .eq('quote_request_id', requestId)
    .eq('status', 'success')
    .order('annual_premium', { ascending: true })

  // Rank them
  if (quotes && quotes.length > 0) {
    for (let i = 0; i < quotes.length; i++) {
      await supabase
        .from('ins_quotes')
        .update({
          rank_position: i + 1,
          rank_score: this.calculateRankScore(quotes[i], quotes),
        })
```

```
        .eq('id', quotes[i].id)
      }
    }

    // Count results
    const { data: allQuotes } = await supabase
      .from('ins_quotes')
      .select('status')
      .eq('quote_request_id', requestId)

    await supabase
      .from('ins_quote_requests')
      .update({
        status: 'complete',
        carriers_succeeded: allQuotes?.filter(q => q.status === 'success').length ||
        carriers_failed: allQuotes?.filter(q => ['error', 'timeout'].includes(q.stat
        completed_at: new Date().toISOString(),
        first_result_at: quotes?.[0]?.completed_at,
        expires_at: new Date(Date.now() + 4 * 60 * 60 * 1000).toISOString(), // 4 ho
      })
      .eq('id', requestId)

    // Final progress publish
    await this.publishProgress(requestId)
  }

  /**
   * Score compuesto para ranking
   */
  private calculateRankScore(quote: any, allQuotes: any[]): number {
    const premiums = allQuotes.map(q => q.annual_premium).filter(Boolean)
    const minPremium = Math.min(...premiums)
    const maxPremium = Math.max(...premiums)

    // Price score: 0-40 (lower is better)
    const priceRange = maxPremium - minPremium || 1
    const priceScore = 40 * (1 - (quote.annual_premium - minPremium) / priceRange)

    // Coverage score: 0-30 (more coverages = better)
    const coverageCount = (quote.coverages || []).length
    const maxCoverages = Math.max(...allQuotes.map(q => (q.coverages || []).length))
    const coverageScore = maxCoverages > 0 ? 30 * (coverageCount / maxCoverages) : 1

    // Carrier rating: 0-20 (based on market share as proxy)
```

```
  const carrierScore = 20 // TODO: Lookup from ins_carriers

  // Speed score: 0-10 (faster = better)
  const speedScore = quote.duration_ms ? 10 * Math.max(0, 1 - quote.duration_ms /

  return Math.round((priceScore + coverageScore + carrierScore + speedScore) * 100
}

// Helper methods...
private extractRiskFields(line: string, riskData: Record<string, any>) {
  if (line === 'auto') {
    return {
      vehicle_brand: riskData.brand,
      vehicle_model: riskData.model,
      vehicle_year: riskData.year,
      vehicle_version: riskData.version,
      vehicle_use: riskData.use || 'particular',
      coverage_type: riskData.coverage || 'amplia',
    }
  }
  // Add more lines...
  return {}
}

private async getAvailableCarriers(tenantId: string, insuranceLine: string) {
  const { data } = await supabase
    .from('ins_carrier_credentials')
    .select('carrier_id, ins_carriers(*)')
    .eq('tenant_id', tenantId)
    .eq('is_active', true)

  return (data || [])
    .map(d => d.ins_carriers)
    .filter(c => c.is_active && c.supported_lines.includes(insuranceLine) && c.hea
}

private async getDecryptedCredentials(tenantId: string, carrierId: string) {
  // TODO: Implement AES-256-GCM decryption
  const { data } = await supabase
    .from('ins_carrier_credentials')
    .select('encrypted_username, encrypted_password, encryption_iv, agent_number')
    .eq('tenant_id', tenantId)
    .eq('carrier_id', carrierId)
    .single()
```

```
  if (!data) throw new Error('No credentials found')

  return {
    username: this.decrypt(data.encrypted_username, data.encryption_iv),
    password: this.decrypt(data.encrypted_password, data.encryption_iv),
    agent_number: data.agent_number,
  }
}

private decrypt(encrypted: string, iv: string): string {
  const key = Buffer.from(process.env.CREDENTIAL_ENCRYPTION_KEY!, 'hex')
  const decipher = crypto.createDecipheriv('aes-256-gcm', key, Buffer.from(iv, 'he
  // Note: Need to handle auth tag properly in production
  let decrypted = decipher.update(encrypted, 'hex', 'utf8')
  return decrypted
}

private async saveQuoteResult(requestId: string, carrier: any, result: any) {
  await supabase
    .from('ins_quotes')
    .update({
      status: result.status || 'success',
      annual_premium: result.annual_premium,
      monthly_premium: result.monthly_premium,
      deductible_amount: result.deductible_amount,
      deductible_percentage: result.deductible_percentage,
      coverages: result.coverages,
      policy_number_preview: result.quote_number || result.policy_number_preview,
      valid_until: result.valid_until,
      pdf_url: result.pdf_url,
      screenshot_url: result.screenshot_url,
      duration_ms: result.duration_ms,
      completed_at: new Date().toISOString(),
    })
    .eq('quote_request_id', requestId)
    .eq('carrier_id', carrier.id)
}

private async markQuoteFailed(requestId: string, carrierId: string, errorMessage:
  await supabase
    .from('ins_quotes')
    .update({
      status: 'error',
```

```
        error_message: errorMessage,
        completed_at: new Date().toISOString(),
      })
      .eq('quote_request_id', requestId)
      .eq('carrier_id', carrierId)
  }

  private async markQuoteSkipped(requestId: string, carrierId: string, reason: strin
    await supabase
      .from('ins_quotes')
      .update({
        status: 'error',
        error_message: `Skipped: ${reason}`,
        error_type: reason,
        completed_at: new Date().toISOString(),
      })
      .eq('quote_request_id', requestId)
      .eq('carrier_id', carrierId)
  }

  private async cacheResult(carrierId: string, input: QuoteRequestInput, result: any
    const cacheKey = this.buildCacheKey(carrierId, input)
    const ttlHours = parseInt(process.env.QUOTE_CACHE_TTL_HOURS || '4')

    await supabase
      .from('ins_quote_cache')
      .upsert({
        cache_key: cacheKey,
        carrier_id: carrierId,
        quote_data: result,
        expires_at: new Date(Date.now() + ttlHours * 60 * 60 * 1000).toISOString(),
      })
  }
}
```

PARTE 5: INTEGRACIÓN WHATSAPP

Capítulo 17: Flujo Conversacional de Cotización por WhatsApp

17.1 Flujo de conversación (estado por estado)

```typescript
```

```typescript
// src/lib/insurance/whatsapp-insurance-flow.ts

import { QuoteOrchestrator } from './quote-orchestrator'

/**
 * Estados del flujo de cotización por WhatsApp
 *
 * IDLE → DETECTING_INTENT → COLLECTING_DATA → VALIDATING → QUOTING → DELIVERING → F
 */

type InsuranceFlowState =
  | 'idle'
  | 'detecting_intent'        // ¿Quiere cotizar? ¿Qué tipo?
  | 'collecting_line'          // ¿Auto, vida, GMM, hogar?
  | 'collecting_vehicle'       // Marca, modelo, año
  | 'collecting_driver'        // Nombre, edad, CP
  | 'collecting_coverage'      // Tipo de cobertura
  | 'confirming_data'          // ¿Todo correcto?
  | 'quoting'                  // Cotizando...
  | 'delivering_results'       // Entregando resultados
  | 'follow_up'               // ¿Te interesa alguna?

interface ConversationState {
  state: InsuranceFlowState
  insuranceLine?: string
  collectedData: Record<string, any>
  missingFields: string[]
  quoteRequestId?: string
}

// System prompt para el AI extractor
const INSURANCE_EXTRACTION_PROMPT = `
Eres un asistente de seguros mexicano experto. Tu trabajo es extraer datos para coti

REGLAS:
1. Sé amigable, usa español mexicano natural (tutea al cliente)
2. Extrae TODOS los datos que puedas del mensaje del usuario
3. Si faltan datos, pregunta de forma conversacional (no como formulario)
4. Nunca pidas todos los datos de golpe — máximo 2-3 preguntas a la vez
5. Usa emojis moderadamente
6. Si el usuario da datos ambiguos, confirma

DATOS REQUERIDOS POR TIPO:
```

```
Auto:
- marca (string)
- modelo (string)
- año (integer 2000-2027)
- version (string, opcional)
- uso: "particular" | "comercial"
- cobertura: "amplia" | "limitada" | "rc"
- cp (string, 5 dígitos)
- nombre_completo (string)
- fecha_nacimiento (string DD/MM/YYYY)
- genero: "M" | "F"

Responde SIEMPRE en formato JSON:
{
  "intent": "quote_insurance" | "check_policy" | "file_claim" | "general_question",
  "insurance_line": "auto" | "vida" | "gastos_medicos" | "hogar" | null,
  "extracted_data": { ... datos que pudiste extraer ... },
  "missing_fields": ["campo1", "campo2"],
  "next_question": "texto amigable para preguntar los datos faltantes",
  "confidence": 0.0-1.0
}
`

export class WhatsAppInsuranceFlow {
  private orchestrator = new QuoteOrchestrator()

  /**
   * Procesar un mensaje de WhatsApp entrante
   */
  async processMessage(
    tenantId: string,
    conversationId: string,
    contactId: string,
    message: string,
    currentState?: ConversationState
  ): Promise<{ reply: string; newState: ConversationState; buttons?: any[] }> {

    const state = currentState || { state: 'idle', collectedData: {}, missingFields:

    // Si estamos en estado quoting, verificar progreso
    if (state.state === 'quoting' && state.quoteRequestId) {
      return this.handleQuotingState(state)
    }
```

```
  // Extraer datos del mensaje con AI
  const extraction = await this.extractInsuranceData(message, state)

  // Merge datos nuevos con los existentes
  const mergedData = { ...state.collectedData, ...extraction.extracted_data }
  const missingFields = extraction.missing_fields

  // Si tenemos todos los datos → confirmar y cotizar
  if (missingFields.length === 0 && extraction.insurance_line) {
    if (state.state !== 'confirming_data') {
      // Mostrar resumen y pedir confirmación
      const summary = this.buildDataSummary(extraction.insurance_line, mergedData)
      return {
        reply: `📋 Perfecto, tengo todos los datos. Confirma que todo esté correct
        newState: {
          state: 'confirming_data',
          insuranceLine: extraction.insurance_line,
          collectedData: mergedData,
          missingFields: [],
        },
        buttons: [
          { id: 'confirm_yes', title: '✅ Sí, cotiza' },
          { id: 'confirm_edit', title: '✏️ Corregir algo' },
        ]
      }
    } else {
      // El usuario confirmó → lanzar cotización
      return this.launchQuoting(tenantId, conversationId, contactId, extraction.in
    }
  }

  // Si faltan datos → preguntar
  return {
    reply: extraction.next_question,
    newState: {
      state: missingFields.length <= 2 ? 'collecting_coverage' : 'collecting_vehic
      insuranceLine: extraction.insurance_line,
      collectedData: mergedData,
      missingFields: missingFields,
    }
  }
}
```

```
/**
 * Lanzar la cotización multi-carrier
 */
private async launchQuoting(
  tenantId: string,
  conversationId: string,
  contactId: string,
  insuranceLine: string,
  data: Record<string, any>
) {
  const quoteRequestId = await this.orchestrator.executeQuote({
    tenantId,
    contactId,
    conversationId,
    insuranceLine: insuranceLine as any,
    clientData: {
      name: data.nombre_completo,
      phone: data.telefono,
      email: data.email,
      rfc: data.rfc,
      birthdate: data.fecha_nacimiento,
      gender: data.genero,
      zipCode: data.cp,
    },
    riskData: data,
    source: 'whatsapp',
  })

  return {
    reply: `⏳ ¡Listo! Estoy cotizando con *15+ aseguradoras* simultáneamente.\n\n
    newState: {
      state: 'quoting' as InsuranceFlowState,
      insuranceLine,
      collectedData: data,
      missingFields: [],
      quoteRequestId,
    }
  }
}

/**
 * Construir resumen legible de los datos
 */
private buildDataSummary(line: string, data: Record<string, any>): string {
```

```
  if (line === 'auto') {
    return [
      `🚗 *Vehículo*: ${data.marca} ${data.modelo} ${data.año}`,
      data.version ? `   Versión: ${data.version}` : null,
      `   Uso: ${data.uso === 'particular' ? 'Particular' : 'Comercial'}`,
      `📍 *CP*: ${data.cp}`,
      `👤 *Conductor*: ${data.nombre_completo}`,
      `   Nacimiento: ${data.fecha_nacimiento}`,
      `   Género: ${data.genero === 'M' ? 'Masculino' : 'Femenino'}`,
      `🛡️ *Cobertura*: ${data.cobertura === 'amplia' ? 'Amplia' : data.cobertura =
    ].filter(Boolean).join('\n')
  }
  return JSON.stringify(data, null, 2)
}

/**
 * Formatear resultados para WhatsApp (progressive delivery)
 */
formatProgressiveResults(progress: any): string {
  const { carriersTotal, carriersCompleted, carriersFailed, results } = progress

  if (carriersCompleted === 0) {
    return `⏳ Cotizando... (0 de ${carriersTotal} aseguradoras)`
  }

  // Sort by premium
  const sorted = results.sort((a: any, b: any) => (a.premium || Infinity) - (b.pre

  let message = `📊 *${carriersCompleted} de ${carriersTotal} cotizaciones listas*

  sorted.forEach((r: any, i: number) => {
    const medal = i === 0 ? '🥇' : i === 1 ? '🥈' : i === 2 ? '🥉' : '▪️'
    message += `${medal} *${r.carrierName}*: $${r.premium?.toLocaleString('es-MX')
  })

  if (carriersCompleted < carriersTotal) {
    const remaining = carriersTotal - carriersCompleted - carriersFailed
    message += `\n⏳ _${remaining} aseguradoras aún cotizando..._`
  } else {
    message += `\n✅ *Cotización completa*`
    if (carriersFailed > 0) {
      message += ` (${carriersFailed} aseguradoras no disponibles)`
    }
    message += `\n\n¿Te interesa alguna? Puedo darte más detalles de cualquiera.`
```

```
  }

  return message
}

/**
 * Formatear comparativa final detallada
 */
formatFinalComparison(quotes: any[]): string {
  const sorted = quotes
    .filter(q => q.status === 'success')
    .sort((a, b) => a.annual_premium - b.annual_premium)

  let msg = `🏆 *COMPARATIVA DE SEGUROS*\n`
  msg += `━━━━━━━━━━━━━━━━━━━━━━\n\n`

  sorted.slice(0, 5).forEach((q, i) => {
    const medal = ['🥇', '🥈', '🥉', '4️⃣', '5️⃣'][i]
    msg += `${medal} *${q.carrier_name}*\n`
    msg += `   💰 Prima anual: *$${q.annual_premium.toLocaleString('es-MX')}*\n`
    if (q.monthly_premium) {
      msg += `   📅 Mensual: $${q.monthly_premium.toLocaleString('es-MX')}\n`
    }
    if (q.deductible_percentage) {
      msg += `   🔧 Deducible: ${q.deductible_percentage}%\n`
    }
    msg += `\n`
  })

  msg += `💡 _Responde con el número (1-5) para ver detalle completo y contratar._

  return msg
}

// AI extraction helper (uses existing LLM infrastructure)
private async extractInsuranceData(message: string, state: ConversationState) {
  // Call OpenRouter with the extraction prompt
  // This integrates with the existing atiende.ai LLM infrastructure
  const response = await fetch('https://openrouter.ai/api/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.OPENROUTER_API_KEY}`,
    },
```

```
      body: JSON.stringify({
        model: 'google/gemini-2.5-flash',
        messages: [
          { role: 'system', content: INSURANCE_EXTRACTION_PROMPT },
          { role: 'user', content: `Estado actual: ${JSON.stringify(state)}\n\nMensa
        ],
        response_format: { type: 'json_object' },
        temperature: 0.1,
      }),
    })

    const data = await response.json()
    return JSON.parse(data.choices[0].message.content)
  }

  private async handleQuotingState(state: ConversationState) {
    // Check Redis for latest progress
    // Return progress update
    return {
      reply: '⏳ Aún cotizando, te aviso en cuanto tenga resultados...',
      newState: state,
    }
  }
}
```

PARTE 6: DASHBOARD Y CRM DE SEGUROS

Capítulo 21: Páginas del Dashboard

21.1 Estructura de páginas (Next.js App Router)

```
src/app/(dashboard)/insurance/
├── page.tsx                    → Dashboard principal de seguros
├── quotes/
│   ├── page.tsx                → Lista de cotizaciones
│   ├── [id]/page.tsx           → Detalle de cotización con comparativa
│   └── new/page.tsx            → Nueva cotización manual
├── policies/
│   ├── page.tsx                → Lista de pólizas activas
```

```
│   ├── [id]/page.tsx            → Detalle de póliza
│   └── renewals/page.tsx        → Pólizas próximas a renovar
├── payments/
│   ├── page.tsx                  → Tracking de pagos
│   └── overdue/page.tsx         → Pagos vencidos
├── claims/
│   ├── page.tsx                  → Lista de siniestros
│   ├── [id]/page.tsx            → Detalle de siniestro
│   └── new/page.tsx             → Registrar nuevo siniestro
├── carriers/
│   ├── page.tsx                  → Mis aseguradoras conectadas
│   ├── connect/page.tsx         → Conectar nueva aseguradora
│   └── health/page.tsx          → Health status de portales
└── settings/
    └── page.tsx                  → Configuración del módulo seguros
```

21.2 Dashboard principal — métricas clave

```typescript
// src/app/(dashboard)/insurance/page.tsx

// Métricas que muestra:
// 1. Cotizaciones hoy / esta semana / este mes
// 2. Pólizas activas totales
// 3. Pólizas por renovar (próximos 30 días) — ALERTA
// 4. Pagos vencidos — ALERTA
// 5. Comisiones del mes
// 6. Carriers conectados y su health status
// 7. Tiempo promedio de cotización
// 8. Tasa de conversión (cotización → póliza)
// 9. Top carriers por volumen
// 10. Gráfica de cotizaciones por día
```

PARTE 7: BACKGROUND AGENTS

Capítulo 25: Agents Autónomos (Cron Jobs)

25.1 Renewal Agent

```typescript
```

```
// src/lib/insurance/agents/renewal-agent.ts

/**
 * CRON: Ejecuta diario a las 8:00 AM
 * Detecta pólizas próximas a vencer y ejecuta acciones
 */

export async function renewalAgent() {
  // 1. Obtener pólizas que vencen en los próximos 30 días
  const { data: policies } = await supabase
    .from('ins_policies_near_renewal')
    .select('*')

  for (const policy of policies || []) {
    const daysLeft = policy.days_to_renewal

    // 30 días: Primera notificación
    if (daysLeft === 30 && !policy.renewal_notification_sent) {
      await sendWhatsAppRenewalReminder(policy, '30_days')
      await supabase.from('ins_policies').update({ renewal_notification_sent: true }
    }

    // 15 días: Re-cotizar automáticamente
    if (daysLeft === 15 && !policy.renewal_quote_id) {
      const quoteId = await autoRequote(policy)
      await supabase.from('ins_policies').update({ renewal_quote_id: quoteId }).eq('
      await sendWhatsAppRenewalWithQuotes(policy, quoteId)
    }

    // 7 días: Urgente
    if (daysLeft === 7) {
      await sendWhatsAppRenewalUrgent(policy)
    }

    // 1 día: Último aviso
    if (daysLeft === 1) {
      await sendWhatsAppRenewalFinal(policy)
    }
  }
}
```

25.2 Payment Checker Agent

typescript

```
// src/lib/insurance/agents/payment-agent.ts

/**
 * CRON: Ejecuta cada 6 horas
 * Accede a portales de aseguradoras y verifica status de pagos
 */

export async function paymentCheckerAgent() {
  // 1. Obtener todas las pólizas activas con pagos pendientes
  const { data: pendingPayments } = await supabase
    .from('ins_policy_payments')
    .select('*, ins_policies(*, ins_carriers(*), contacts(*))')
    .in('status', ['pending', 'overdue'])
    .lt('due_date', new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString())

  // 2. Agrupar por carrier para minimizar logins
  const byCarrier = groupBy(pendingPayments, p => p.ins_policies.carrier_id)

  for (const [carrierId, payments] of Object.entries(byCarrier)) {
    // 3. Login una vez al portal y verificar todos los pagos de ese carrier
    try {
      const results = await checkPaymentsInPortal(carrierId, payments)

      for (const result of results) {
        if (result.paid) {
          await supabase.from('ins_policy_payments').update({
            status: 'paid',
            paid_date: result.paid_date,
            verified_from_portal: true,
            portal_check_at: new Date().toISOString(),
            portal_reference: result.reference,
          }).eq('id', result.paymentId)
        } else if (result.overdue && !result.notified) {
          // Enviar recordatorio por WhatsApp
          await sendPaymentReminder(result)
        }
      }
    } catch (err) {
      console.error(`Payment check failed for carrier ${carrierId}:`, err)
    }
  }
}
```

PARTE 8: FASES DE IMPLEMENTACIÓN

Reglas para Claude Code

1. **Siempre pregunta antes de actuar** — No asumas API keys ni credenciales
2. **Un paso a la vez** — Sigue las fases EN ORDEN. No saltes fases.
3. **Testing primero** — Cada fase tiene checkpoints verificables
4. **Este módulo se AGREGA a atiende.ai** — No reemplazas nada existente
5. **Archivos van en** `src/lib/insurance/` para backend, `src/app/(dashboard)/insurance/` para frontend
6. **Workers de Python van en** `workers/` (deploy separado en Railway)
7. **Toda credencial encriptada** — AES-256-GCM, nunca plaintext
8. **Supabase RLS en TODAS las tablas** — Multi-tenant obligatorio
9. **Progressive delivery por WhatsApp** — No esperar a que todas las aseguradoras respondan
10. **Circuit breakers** — Si un carrier falla >50%, skip automático

---

FASE 0: Setup del Módulo de Seguros

Objetivo: Crear la estructura de archivos y dependencias sin romper atiende.ai existente.
Dependencias: atiende.ai base debe estar funcionando **Tiempo estimado**: 30 minutos

Qué pedir al usuario:

- [ ] "¿Tu instancia de atiende.ai está deployada y funcionando?"
- [ ] "¿Tienes acceso a Supabase SQL Editor?"
- [ ] "Dame tu SKYVERN_API_KEY (registrate en skyvern.com si no tienes)"
- [ ] "Dame tu UPSTASH_REDIS_REST_URL y TOKEN"

Archivos a crear:

```
src/lib/insurance/
├── types.ts                          → Tipos TypeScript compartidos
├── constants.ts                      → Constantes (carriers, coverage types, etc)
├── quote-orchestrator.ts             → Motor de cotización multi-carrier
├── whatsapp-insurance-flow.ts        → Flujo conversacional WhatsApp
├── credential-vault.ts               → Encriptación/desencriptación de
credenciales
├── ranking-engine.ts                 → Algoritmo de ranking de cotizaciones
├── cache-manager.ts                  → Gestión de caché de cotizaciones
├── agents/
│   ├── renewal-agent.ts              → Agente de renovaciones
│   ├── payment-agent.ts              → Agente de verificación de pagos
│   └── reconciliation-agent.ts       → Agente de reconciliación
└── utils/
    ├── zip-codes.ts                  → Utilidades de código postal
    ├── vehicle-catalog.ts            → Catálogo AMIS de vehículos
    └── formatters.ts                 → Formateo de moneda, fechas, etc

src/app/(dashboard)/insurance/
├── page.tsx                          → Dashboard principal seguros
├── layout.tsx                        → Layout del módulo seguros
├── quotes/
│   ├── page.tsx
│   └── [id]/page.tsx
├── policies/
│   ├── page.tsx
│   └── [id]/page.tsx
├── carriers/
│   └── page.tsx
└── settings/
    └── page.tsx

src/app/api/insurance/
├── quote/route.ts                    → POST: Iniciar cotización
├── quote/[id]/route.ts               → GET: Status de cotización
├── quote/[id]/stream/route.ts        → GET (SSE): Progressive results
├── carriers/route.ts                 → GET/POST: Gestión de carriers
├── credentials/route.ts              → POST: Guardar credenciales
├── policies/route.ts                 → GET/POST: Gestión de pólizas
├── payments/route.ts                 → GET: Status de pagos
├── webhooks/skyvern/route.ts         → POST: Callbacks de Skyvern
└── cron/
    ├── renewals/route.ts             → Cron: Renovaciones
```

```
    ├── payments/route.ts            → Cron: Verificación pagos
    └── health/route.ts              → Cron: Health check carriers

workers/                              → Python workers (Railway deploy)
├── Dockerfile
├── requirements.txt
├── main.py                           → FastAPI app
├── base_worker.py
├── skyvern_worker.py
├── playwright_workers/
│   ├── qualitas.py
│   ├── gnp.py
│   └── axa.py
└── utils/
    ├── encryption.py
    └── supabase_client.py
```

Checkpoint:

- [ ] `npm run build` pasa sin errores
- [ ] Estructura de carpetas creada correctamente
- [ ] `.env.local` actualizado con nuevas variables (valores placeholder)
- [ ] No se rompió ninguna funcionalidad existente de atiende.ai

---

FASE 1: Base de Datos

Objetivo: Crear todas las tablas del módulo de seguros en Supabase **Dependencias:** Fase 0 **Tiempo estimado:** 20 minutos

Qué pedir al usuario:

- [ ] "Voy a ejecutar el SQL en tu Supabase. ¿Confirmas?"

Qué hacer:

1. Ejecutar TODO el SQL del Capítulo 5 en Supabase SQL Editor
2. Verificar que las 12 tablas se crearon correctamente
3. Verificar que las 3 vistas se crearon
4. Verificar RLS en todas las tablas
5. Verificar los seed data de carriers

Checkpoint:

- [ ] `SELECT count(*) FROM ins_carriers;` retorna 15
- [ ] `SELECT * FROM ins_carrier_health;` funciona (vista)
- [ ] `SELECT * FROM ins_policies_near_renewal;` funciona (vista)
- [ ] RLS habilitado en todas las tablas `ins_*`

FASE 2: Credential Vault

Objetivo: Sistema de encriptación para guardar credenciales de portales **Dependencias**: Fase 1 **Tiempo estimado**: 45 minutos

Qué pedir al usuario:

- [ ] "Genera tu encryption key: `openssl rand -hex 32` y agrégala como CREDENTIAL_ENCRYPTION_KEY en .env"

Archivos a crear: `src/lib/insurance/credential-vault.ts`

Checkpoint:

- [ ] Puede encriptar y desencriptar un username/password
- [ ] Datos encriptados guardados en `ins_carrier_credentials`
- [ ] Datos desencriptados correctamente al leer
- [ ] Test: encrypt("miUsuario") → decrypt → "miUsuario"

FASE 3: Types y Constants

Objetivo: Definir todos los tipos TypeScript y constantes del módulo **Dependencias**: Fase 1 **Tiempo estimado**: 30 minutos

Archivos a crear: `src/lib/insurance/types.ts`, `src/lib/insurance/constants.ts`

Checkpoint:

- [ ] Todos los tipos compilar sin errores
- [ ] Constantes de carriers, coverage types, insurance lines definidas

---

FASE 4: API Routes

Objetivo: Crear todos los endpoints del API de seguros **Dependencias**: Fase 2, 3 **Tiempo estimado**: 2 horas

Archivos a crear: Todos los archivos en `src/app/api/insurance/`

Checkpoint:

- [ ] `POST /api/insurance/credentials` guarda credenciales encriptadas
- [ ] `GET /api/insurance/carriers` retorna lista de carriers con health status
- [ ] `POST /api/insurance/quote` crea un quote request (sin ejecutar aún)
- [ ] `GET /api/insurance/quote/[id]` retorna status del quote
- [ ] Auth requerida en todos los endpoints (Supabase JWT)
- [ ] Rate limiting implementado

FASE 5: Quote Orchestrator

Objetivo: Implementar el motor de cotización multi-carrier completo **Dependencias:** Fase 4 **Tiempo estimado:** 3 horas

Archivos a crear: `src/lib/insurance/quote-orchestrator.ts`

Implementar el código completo del Capítulo 9 con:

- Fan-out a múltiples carriers
- Redis pub/sub para progressive updates
- Circuit breaker por carrier
- Cache de cotizaciones
- Ranking engine

Checkpoint:

- [ ] Puede crear un QuoteRequest y fan-out a carriers (mock mode)
- [ ] Redis recibe progressive updates
- [ ] Circuit breaker funciona (manual test: abrir/cerrar)
- [ ] Cache hit funciona (segunda cotización idéntica es instantánea)

FASE 6: Python Workers Setup

Objetivo: Crear la infraestructura de workers en Python para browser automation **Dependencias:** Fase 5 **Tiempo estimado:** 2 horas

Qué pedir al usuario:

- [ ] "¿Tienes Docker instalado localmente para testing?"
- [ ] "¿Tienes cuenta de Railway?"

Archivos a crear: Todo en `workers/`

- Dockerfile con Playwright + Chrome pre-instalado
- FastAPI app con endpoints /quote, /health
- Base worker class
- Skyvern worker genérico
- Placeholder para Qualitas, GNP, AXA workers

Checkpoint:

- [ ] `docker build -t insurance-workers .` exitoso
- [ ] `docker run insurance-workers` → FastAPI corriendo en :8080
- [ ] `curl localhost:8080/health` retorna 200
- [ ] `curl -X POST localhost:8080/quote` (con mock data) retorna resultado

FASE 7: Skyvern Integration

Objetivo: Integrar Skyvern SDK para browser automation AI **Dependencias**: Fase 6 **Tiempo estimado**: 3 horas

Qué pedir al usuario:

- [ ] "Dame tu SKYVERN_API_KEY"
- [ ] "¿Tienes credenciales de prueba de al menos 1 aseguradora?"

Qué hacer:

1. Implementar SkyvernCarrierWorker completo
2. Testear con un portal real (si hay credenciales)
3. Implementar manejo de CAPTCHA
4. Implementar timeout y retries
5. Screenshot capture y upload a Supabase Storage

Checkpoint:

- [ ] Skyvern puede navegar al login page de una aseguradora
- [ ] (Con credenciales reales) Login exitoso
- [ ] Data extraction schema funciona
- [ ] Screenshots guardados en Supabase Storage

FASE 8: WhatsApp Insurance Flow

Objetivo: Flujo conversacional completo de cotización por WhatsApp **Dependencias**: Fase 5, 7 **Tiempo estimado**: 3 horas

Archivos a crear: `src/lib/insurance/whatsapp-insurance-flow.ts`

Implementar el código del Capítulo 17 con:

- Detección de intent (quiero cotizar seguro)
- Recolección conversacional de datos
- Validación progresiva
- Confirmación de datos
- Lanzamiento de cotización
- Progressive delivery de resultados
- Follow-up

Checkpoint:

- [ ] Enviar "quiero cotizar seguro de auto" → detecta intent correctamente
- [ ] Responde pidiendo datos faltantes (marca, modelo, año)
- [ ] Después de dar todos los datos → muestra resumen y pide confirmación
- [ ] Al confirmar → lanza cotización y muestra "Cotizando con 15+ aseguradoras..."
- [ ] Progressive results llegan por WhatsApp conforme carriers responden

---

FASE 9: Dashboard Frontend

Objetivo: Crear todas las páginas del dashboard de seguros **Dependencias**: Fase 4 **Tiempo estimado**: 4 horas

Archivos a crear: Todos en `src/app/(dashboard)/insurance/`

Páginas prioritarias:

1. Dashboard principal con métricas
2. Lista de cotizaciones con tabla interactiva
3. Detalle de cotización con comparativa visual
4. Gestión de carriers (conectar/desconectar)
5. Lista de pólizas
6. Pagos pendientes/vencidos

Checkpoint:

- [ ] Dashboard muestra métricas reales desde Supabase
- [ ] Tabla de cotizaciones con sort/filter funciona
- [ ] Comparativa visual de cotizaciones muestra cards por carrier
- [ ] Formulario de credenciales de carrier encripta correctamente
- [ ] Responsive design (mobile-first)

FASE 10: Background Agents (Cron Jobs)

Objetivo: Implementar los agentes autónomos de renovación, pagos y reconciliación
Dependencias: Fase 7, 8 **Tiempo estimado**: 3 horas

Archivos a crear: `src/lib/insurance/agents/*` y cron routes

Implementar:

1. Renewal Agent (diario 8am)
2. Payment Checker Agent (cada 6hrs)
3. Reconciliation Agent (diario 10pm)
4. Carrier Health Check (cada hora)

Checkpoint:

- [ ] Vercel Cron configurado para cada agent
- [ ] Renewal agent detecta pólizas próximas a vencer
- [ ] Envía notificaciones WhatsApp correctamente
- [ ] Health check actualiza status de carriers

FASE 11: Deploy a Producción

Objetivo: Deploy completo del módulo de seguros **Dependencias**: Todas las fases anteriores
Tiempo estimado: 2 horas

Qué pedir al usuario:

- [ ] "¿Variables de entorno de producción configuradas en Vercel?"
- [ ] "¿Railway proyecto creado para workers?"

Pasos:

1. Push a GitHub → Vercel auto-deploy

2. Deploy workers a Railway:

```bash
cd workers/
railway login
railway init
railway up
```

3. Configurar variables de entorno en Railway
4. Configurar Vercel Cron jobs
5. Verificar health check de todos los servicios
6. Test end-to-end: WhatsApp → cotización → resultado

Checkpoint:

- [ ] `https://tuapp.vercel.app/insurance` carga correctamente
- [ ] Workers en Railway respondiendo a health checks
- [ ] Test de cotización completo via WhatsApp
- [ ] Cron jobs ejecutando correctamente
- [ ] Sentry reportando errores (sin errores críticos)
- [ ] Primer cotización real exitosa con al menos 1 carrier

POST-DEPLOY: Checklist de validación

- [ ] Cotización multi-carrier funciona con al menos 3 carriers
- [ ] Progressive delivery por WhatsApp funciona
- [ ] CRM crea deals automáticamente
- [ ] Renewal agent detecta pólizas próximas
- [ ] Payment checker verifica pagos
- [ ] Circuit breaker funciona (si un carrier falla, se skipea)
- [ ] Cache funciona (segunda cotización idéntica es rápida)
- [ ] Dashboard muestra datos reales
- [ ] Credenciales están encriptadas en DB
- [ ] RLS funciona (tenant A no ve datos de tenant B)
- [ ] Rate limiting funciona
- [ ] Logging y monitoring activos

NOTAS FINALES

Diferencias clave vs Handle AI

1. **Self-service vs Forward Deployed**: No necesitamos ingenieros on-site. El agente configura sus credenciales en el dashboard.
2. **WhatsApp-first vs Email-first**: Nuestro canal primario es WhatsApp, no email.
3. **Precio 100x menor**: $150-350 USD/mes vs $50-100K USD/año.
4. **México-specific**: Optimizado para AMIS, SEPOMEX, CP mexicanos, RFC, aseguradoras MX.
5. **Integrado en atiende.ai**: No es un producto standalone. Es un módulo/vertical del SaaS existente.

Carriers prioritarios (por market share auto MX)

1. Qualitas — 32.8% (script Playwright custom)
2. GNP — 12.5% (script Playwright custom)
3. AXA — 8.3% (script Playwright custom)
4. HDI — 7.1% (Skyvern)
5. Chubb — 6.8% (Skyvern)
6. BBVA — 5.2% (API directa)
7. Zurich — 4.1% (Skyvern)
8. Mapfre — 3.8% (Skyvern)
9. Atlas — 3.2% (Skyvern)
10. AIG — 2.9% (Skyvern)

Top 10 = ~86.7% del mercado auto mexicano.

Riesgo #1: Portales que cambian

Cada vez que una aseguradora actualiza su portal:

- Scripts Playwright (Tier 2) → se rompen, requieren fix manual
- Skyvern workers (Tier 3) → se auto-adaptan en la mayoría de los casos
- API directa (Tier 1) → nunca se rompe

Mitigación: Health monitoring cada hora, alertas inmediatas, Skyvern como fallback universal.

Riesgo #2: Bloqueo de cuentas

Si una aseguradora detecta acceso automatizado:

- Pueden bloquear las credenciales del agente
- **Mitigación**: User-agent rotation, velocidad humana (delays aleatorios), IP residencial, máximo 20 cotizaciones/hora por carrier

---

FIN DEL BLUEPRINT — VERSIÓN 1.0 — ABRIL 2026